Agenda SOLID

Q: Design a software which can store diff birds

V0

```
class Bird {
    ==
    fly()
    makeSound()
}
```

```
fly() {
    if() {
    ==
    } else if() {
    ==
    }
```

↳ violating SRP
OCP

V1

```
abstract class Bird {
    ==
    abstract fly();
    eat() {=};
    abstract makeSound();
}
```

SRP ✓
OCP ✓

Pigeon
fly() {
=
}

Parrot
fly() {
=
}

Penguin
fly() {
// no fly
}

↳ empty func

throw Errs.

Liskov's Substitution Principle

Object of any child class should be as-is substitutable in a reference variable of parent type, without requiring any code changes.

Bird b = new Pigeon
new Penguin
new Eagle

b.fly() ⟶ error
not doing anything.

(v2)

Abstract class Bird {
≡
eat();
}

abstract class FlyBird{
abstract fly();
}

abstract class NoFlyBird{
}

Pigeon Parrot Penguin

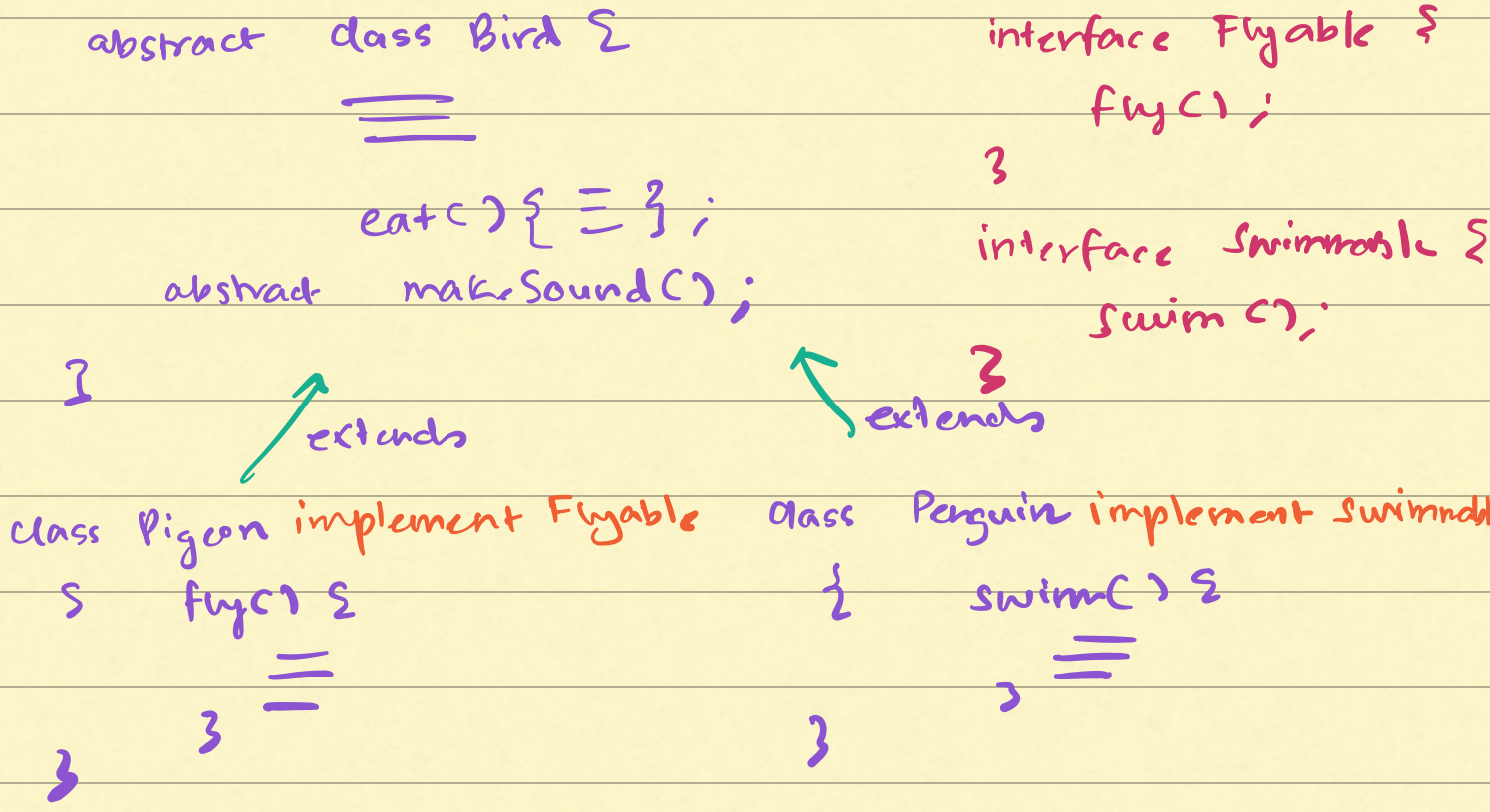
F, F+S, NF+NS, S

n features → 2^n classes → class explosion.

(V3)

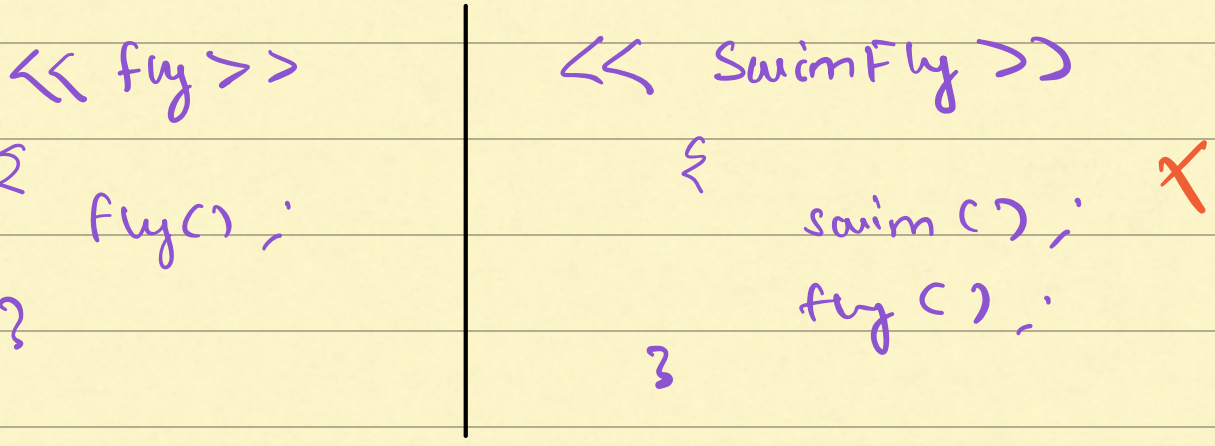
Classify by Entity → class

Classify by Behaviour → Interface



Interface Segregation Principle:

- Some birds can fly
- Some birds can swim
- All birds which can swim can also fly



<< swim >>
{
swim();
}

class — implement fly, swi

→ keep every interface as light as possible

→ Ideally each interface should have 1 function.

Ex:

<<Stack>>
push()
pop()
peek()

→ X It violates ISP

Dependency Inversion Principle

class Bird {

}

interface Flyable,

interface Swimmable

class Pigeon extends Bird implements Flyable {

void fly() {

// low flying

}

}

class Eagle ... {

fly() {

// high flying

}

}

class Parrot ... {

fly() {

// low flying

}

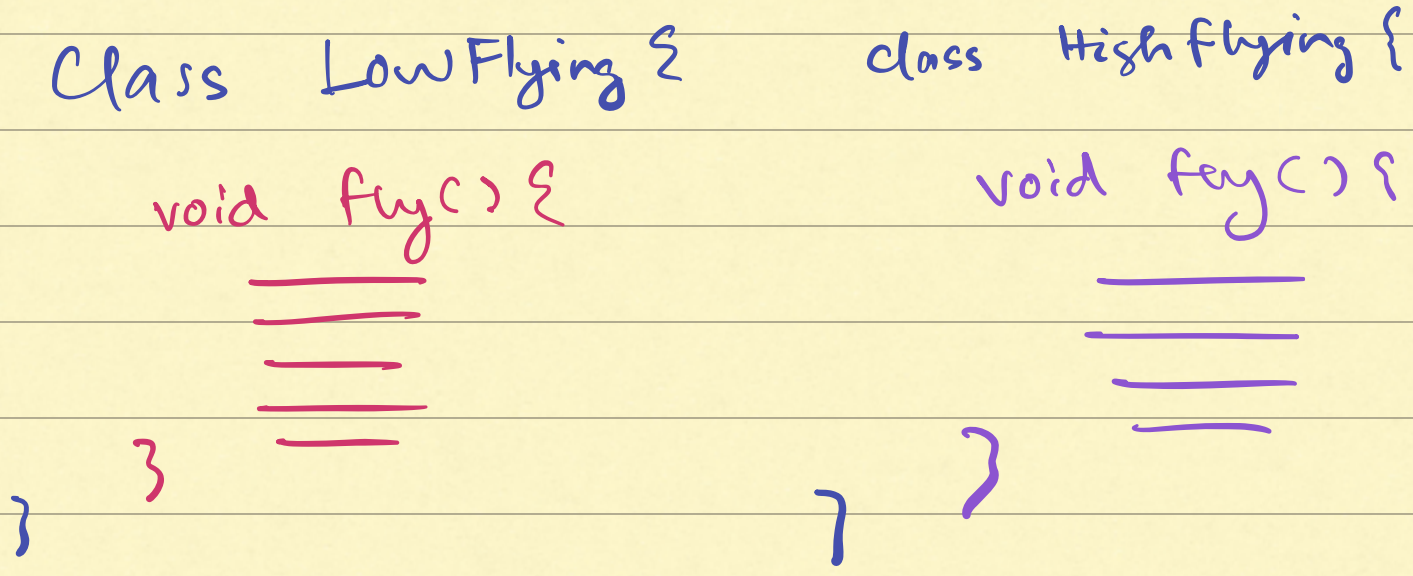
}

class Vulture

fly() {

// high flying

}



- A concrete class should not depend on another concrete class
Instead a concrete Class should depend on interfaces / Abstract class.

Class Pigeon ⟶ class HighFlying ✗

Class Pigeon ⟶ Interface

Class I High Flying

Class II Low Flying